

Step 2 - Order & Inventory Endpoints

Assigned to: **Team Member 2** | Status: Todo list

Goal

Build the ordering flow: a customer creates an order with items, the total is computed server-side, and inventory stock is deducted. Also expose inventory management endpoints for the admin.

Files to create / modify

- **app/schemas/order.py** - OrderItemIn (menu_item_id, quantity >= 1), OrderCreate (list of items), OrderItemOut, OrderOut (with computed total).
- **app/repositories/order_repository.py** - CRUD + queries for orders and order items.
- **app/services/order_service.py** - totals, stock validation, stock deduction, order status transitions.
- **app/api/v1/orders.py** - router with prefix /orders.
- **app/api/v1/inventory.py** - router with prefix /inventory.
- **app/api/v1/__init__.py** - register both routers.

Endpoints to implement

- POST /orders - customer only (current user): creates order + items, computes total, checks stock, deducts stock.
- GET /orders - customers see their own orders only; admins see all.
- GET /orders/{id} - owner or admin only, 404 if missing, 403 if not allowed.
- PATCH /orders/{id}/status - admin only; statuses: pending, confirmed, completed, cancelled.
- GET /inventory - admin only, lists stock per menu item.
- PATCH /inventory/{menu_item_id} - admin only, sets or adds stock (restock).

Hints

- Reject the whole order with 409 if any item is out of stock (rollback, no partial deduction).
- Compute prices from menu_items.price at order time - never trust client prices.
- Commit once, at the end, after all stock updates succeed (single transaction).
- Use selectinload / eager loading for order items to avoid N+1 queries.

Verification

Create a menu item with stock=5, place an order for quantity 2, confirm stock becomes 3, total equals 2 x price. Order 10 more -> expect 409 and stock unchanged (still 3). Complete the flow to 'completed'. A non-owner user must get 403 on GET /orders/{id}. Run the full test suite:

```
backend\venv\Scripts\python.exe -m pytest tests -v
```

Shared Rules

Code conventions

- Layered architecture: schemas -> repositories -> services -> routers. Keep business logic out of routers.
- No comments in code unless truly necessary; follow the style of existing files.
- Routers use APIRouter with explicit tags and response_model.
- Never expose hashed_password or internal fields in responses.
- Auth dependencies: app.api.deps.get_current_user / get_current_admin.

Error handling

- 404 for missing resources, 403 for unauthorized roles, 409 for conflicts.
- Raise HTTPException in services with clear detail messages.
- Validate with pydantic schemas (min_length, ge=0 for money/quantities).

Testing & delivery

- Add tests under tests/ (see tests/test_auth.py as template, incl. cleanup of test rows).
- All tests must pass: backend\venv\Scripts\python.exe -m pytest tests -v
- Commit and push your branch, then report the endpoints you implemented.

Generated for the Smart Canteen team - Phase 2 workstreams.